

CSE233 Week 14: Pointers and Dynamic Arrays

A pointer, in the simplest terms, is a variable that stores a memory address instead of a value. Pointers can be used to reference other variables, or create new memory in an area called the heap.

Basic Pointer Examples

A pointer is declared by putting an asterisk/star `*` after the type name and before the variable name.

```
int *ptr;
```

The `nullptr` value is used to represent a pointer that references nothing. A pointer with the value of `nullptr` is considered `false` in Boolean expressions, just like `0` (most of the time, the literal value of `nullptr` is `0`).

```
int *ptr = nullptr;
if (ptr) {
    std::cout << "ptr is not null\n";
}
else {
    std::cout << "ptr is null\n";
}
```

Pointers store memory addresses. One way to use a pointer is to assign it the address of an existing variable. The ampersand `&` is used access the memory address of a variable:

```
int x = 5;
int *ptr = &x;
```

To access the value of the variable being referenced by the pointer, you put the star `*` in front of the pointer name to dereference it.

```
int x = 5;
int *ptr = &x;
// prints: 5
```

```
std::cout << *ptr << std::endl;
```

Dereferencing also lets you change the value of the variable the pointer is referencing:

```
int x = 5;
int *ptr = &x;
// prints: 5
std::cout << *ptr << std::endl;
*ptr = 10;
// prints: 10
std::cout << *ptr << std::endl;
// prints: 10
std::cout << x << std::endl;
```

Like with pass-by-reference parameters, changes to the pointer affect the variable it is referencing.

Pointers can also be used to create new memory. This memory is allocated in a part of the program memory called the heap. Memory allocated in the heap must be manually freed by the programmer. Pointers that create new memory are typically called raw pointers.

```
// raw pointers are allocated memory
// using the new operator
int *ptr = new int;
// can be used like other pointers
*ptr = 10;
std::cout << *ptr << std::endl;
// use delete to free memory used by raw pointer
delete ptr;
```

Dynamic Arrays

Previously, we could only declare arrays with sizes that are known at compile time. Using pointers and heap allocated memory, we can create dynamic arrays that have a size determined while the program is running.

```
int size;
std::cout << "Enter size: ";
std::cin >> size;

int *dynamicArray = new int[size];
std::cout << "Enter " << size << " numbers\n";
for (int i = 0; i < size; ++i) {
    std::cout << "Number " << i + 1 << ": ";
    // dynamic arrays are not dereferenced, just use the index
    std::cin >> dynamicArray[i];
}
```

```

}

std::cout << "You entered\n";
for (int i = 0; i < size; ++i) {
    if (i != 0) std::cout << ", ";
    std::cout << dynamicArray[i];
}
std::cout << "\n";

// use delete with empty brackets
// to free a dynamic array
delete[] dynamicArray;

```

Pointers to Objects

We've seen that objects use the dot operator to access their fields. However, when we have a pointer to an object, that changes slightly. Instead of the dot operator, we use the array operator `->` to access object members:

```

#include <iostream>
#include <string>

struct Student {
    std::string name;
    char grade;
};

void printStudent(Student* student) {
    std::cout << "Name: " << student->name << std::endl;
    std::cout << "Grade: " << student->grade << std::endl;
}

int main() {
    Student *student = new Student;
    student->name = "John Doe";
    student->grade = 'A';
    printStudent(student);
    delete student;
    return 0;
}

```

The same goes for member functions:

```

#include <string>

int main() {
    std::string* str = new std::string("Hello, World");
    str->append("!");
    std::string* substr = new std::string(str->substr(0, 7));
    // still need to dereference to use operators
    *substr += "John Doe!";
}

```

```

std::cout << *str << std::endl;
std::cout << *substr << std::endl;

delete str;
delete substr;

return 0;
}

```

Pointers as Parameters

As seen in the struct example, pointers can be used as parameters to functions. Pointers behave very similar to pass-by-reference when used as function parameters. The primary difference is that in the function, the pointer parameter must be dereferenced, whereas pass-by-reference parameters are used like they are normal variables. Additionally, when the function is called, we either need to pass a pointer as the argument or pass the address of the variable being used as an argument (using &). Below is an example:

```

#include <cassert>

namespace ref {
    void swap(int& x, int& y) {
        int temp = x;
        x = y;
        y = temp;
    }
}

namespace ptr {
    void swap(int* x, int* y) {
        int temp = *x;
        *x = *y;
        *y = *temp;
    }
}

int main() {
    int x = 10;
    int y = 4;
    // with pass-by-ref, we use x and y as parameters
    ref::swap(x, y);
    assert(x == 4);
    assert(y == 10);

    // with pointer params, we need to use & to get the addresses
    ptr::swap(&x, &y);
    assert(x == 10);
    assert(y == 4);
}

```

```
    int *xPtr = &x;
    int *yPtr = &y;
    // or, we can directly use pointers as the arguments
    ptr::swap(xPtr, yPtr);

    return 0;
}
```

Pointers as Return Values

Pointers can also be the return value of a function. Don't return pointers to variables from a function. When the function ends, local variables are deleted and the pointer may become invalid. Returning raw pointers and dynamic arrays is possible, but it becomes the responsibility of the calling code to call delete on the pointer that is returned when it is no longer needed.

```
// don't do this
int* pointerToLocal() {
    int x = 10;
    int *ptr = &x;
    // this pointer may become invalid after the function ends
    // don't return pointers that point to local variables
    return ptr;
}

// this is fine, but client has to delete the pointer
int* getRawPointer(int val) {
    int *ptr = new int;
    *ptr = val;
    return ptr;
}

// this is also fine. again, calling code must clean up
int* getDynamicArray(int size) {
    int *darray = new int[size];
    return darray;
}

int main() {
    int *rawPtr = getRawPointer(10);
    delete rawPtr;
    int *darray = getDynamicArray(32);
    delete[] darray;

    return 0;
}
```

Conclusion

Pointers are variables that are used to store memory addresses instead of values. Accessing the value referenced by a pointer requires the

pointer to be dereferenced. Pointers can be assigned the memory address of existing variables, or used to create raw pointers using the `new` operator. Pointers created with `new` should always have a corresponding `delete` to free the memory created by the pointer. Unlike static arrays (declared like `int array[100]`), where the size must be known when the program is compiled, pointers can also be used to create dynamic arrays which can be created with a size that isn't known until the program is run. Dynamic arrays are freed using the `delete[]` operator. Pointers can be used as parameters to functions as well as return values from functions.